

AI-Powered Travel Recommendation & Tourist Services Platform

System Design Architecture & Development Blueprint

Project Type: AI-powered Travel Recommendation, Trip Planning & Tourist Service Platform

Primary AI Model: Qwen3 (Open-Source)

Frontend: React + TypeScript + Tailwind CSS

Backend: FastAPI + Python

Database: PostgreSQL + pgvector

AI Architecture: LLM + RAG + Tool Calling + Recommendation Engine

Deployment: Docker + Cloud Infrastructure

1. Project Overview

The proposed system is an **AI-powered travel recommendation and tourist service platform** designed to help travelers discover destinations, generate personalized itineraries, find hotels and restaurants, discover tourist activities, estimate trip costs, and manage their complete travel plans.

The system combines:

- Open-source LLM
- Retrieval-Augmented Generation (RAG)
- Real-time travel APIs
- Maps and routing
- Weather information
- Recommendation algorithms
- Constraint-based itinerary optimization
- Tourist service provider information
- Personalized user profiles

The primary objective is to provide **reliable, personalized, and context-aware travel recommendations instead of simply generating text using an LLM.**

2. Problem Statement

Traditional travel applications usually require users to manually search across multiple platforms for:

- Destinations
- Hotels

- Restaurants
- Tourist attractions
- Weather
- Routes
- Transportation
- Activities
- Prices
- Travel schedules

This creates fragmented information and makes trip planning time-consuming.

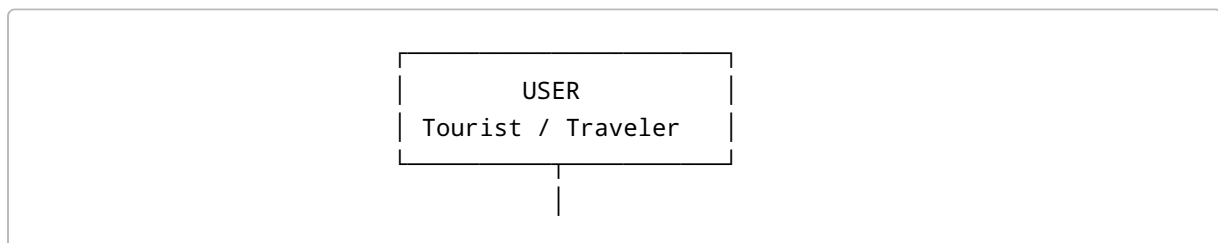
The proposed system solves this problem by providing a centralized AI-powered travel assistant that can understand user preferences and combine information from multiple sources to create an optimized travel plan.

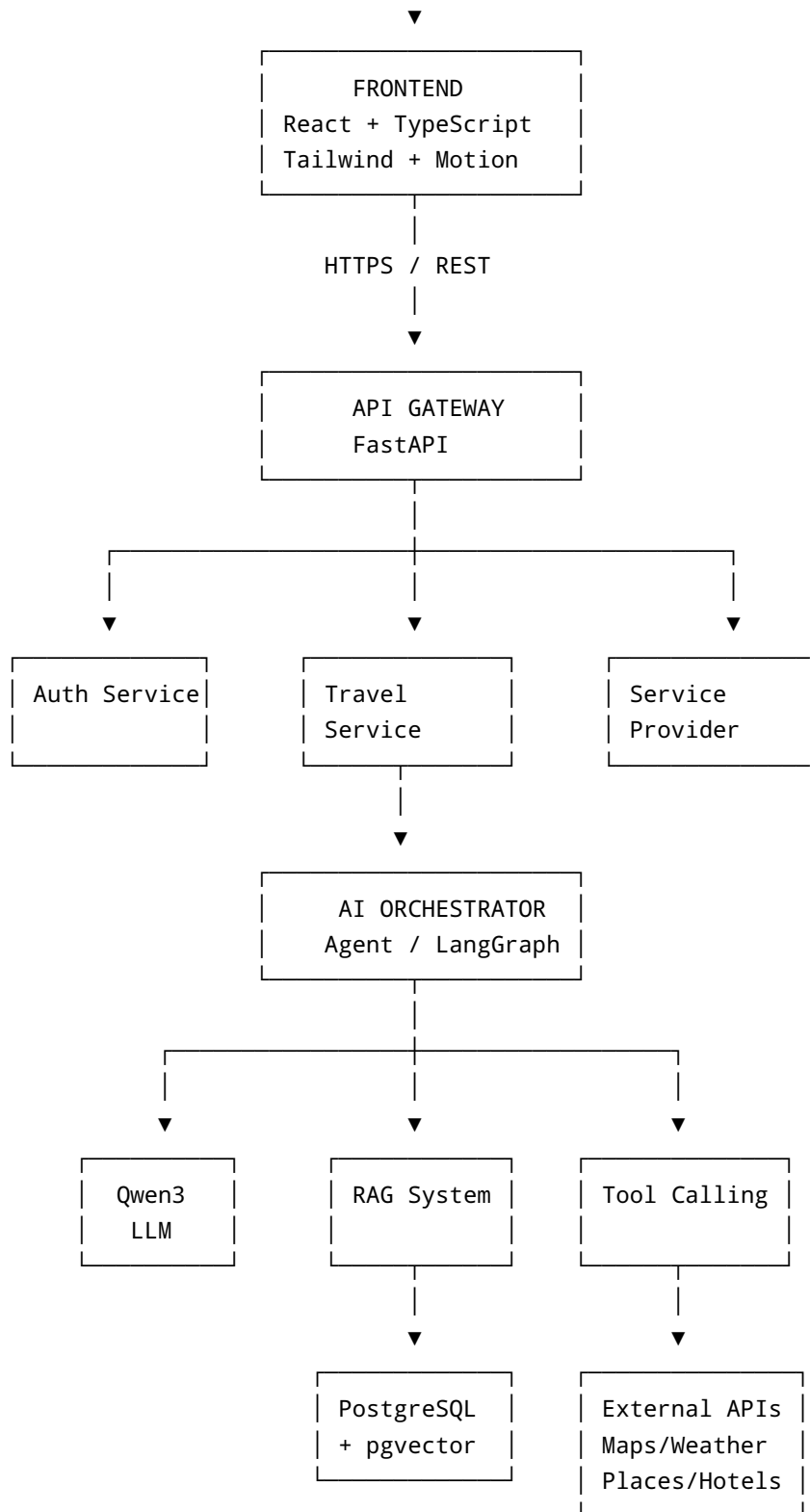
3. Main Objectives

The system should:

1. Recommend destinations based on user preferences.
2. Generate personalized travel itineraries.
3. Recommend hotels and accommodations.
4. Recommend restaurants.
5. Recommend tourist attractions and activities.
6. Calculate approximate travel costs.
7. Consider weather conditions.
8. Calculate routes and travel times.
9. Provide real-time information through external APIs.
10. Use RAG for factual travel knowledge.
11. Use tool calling to retrieve live information.
12. Validate AI-generated itineraries.
13. Allow users to save and manage trips.
14. Provide a platform for tourist service providers.
15. Provide an administrator dashboard.

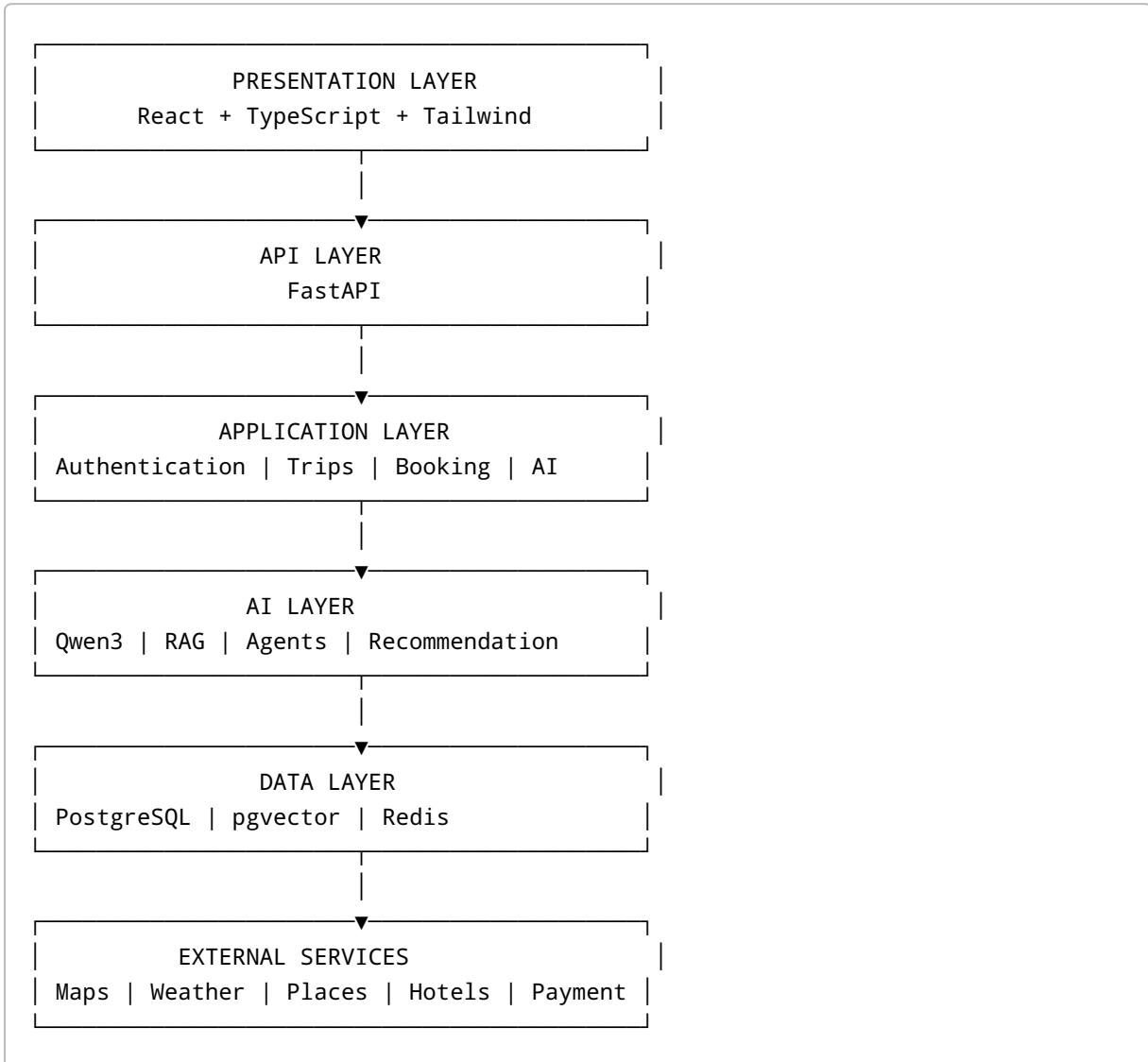
4. High-Level System Architecture





5. Core Architecture Layers

The system should be divided into the following layers.



6. Frontend Architecture

Technology

Use:

- React
- TypeScript

- Vite
- Tailwind CSS
- Motion / Framer Motion
- React Router
- TanStack Query
- Zustand or Context API

Main pages

```
/
├─ Landing Page
├─ Login
├─ Register
├─ Explore
├─ Destination Details
├─ AI Trip Planner
├─ Trip Details
├─ Hotels
├─ Restaurants
├─ Activities
├─ Saved Trips
├─ User Profile
├─ AI Assistant
└─ Service Provider Dashboard
```

Frontend structure

```
frontend/
├─
├─ src/
│   ├── components/
│   │   ├── Navbar/
│   │   ├── Hero/
│   │   ├── DestinationCard/
│   │   ├── HotelCard/
│   │   ├── RestaurantCard/
│   │   ├── ActivityCard/
│   │   ├── Itinerary/
│   │   ├── Map/
│   │   ├── AIChat/
│   │   └─ BudgetChart/
│   ├── pages/
│   └─ layouts/
```

```
├── hooks/
├── services/
├── stores/
├── types/
├── utils/
├── data/
└── package.json
```

7. Backend Architecture

Use **FastAPI** as the primary backend framework.

```

backend/
├── app/
│   ├── main.py
│   ├── api/
│   │   ├── auth.py
│   │   ├── users.py
│   │   ├── trips.py
│   │   ├── destinations.py
│   │   ├── hotels.py
│   │   ├── restaurants.py
│   │   ├── activities.py
│   │   └── ai.py
│   ├── services/
│   │   ├── auth_service.py
│   │   ├── travel_service.py
│   │   ├── recommendation_service.py
│   │   ├── itinerary_service.py
│   │   └── booking_service.py
│   └── ai/
│       ├── llm.py
│       ├── agent.py
│       ├── rag.py
│       ├── embeddings.py
│       ├── tools.py
│       └── prompts.py

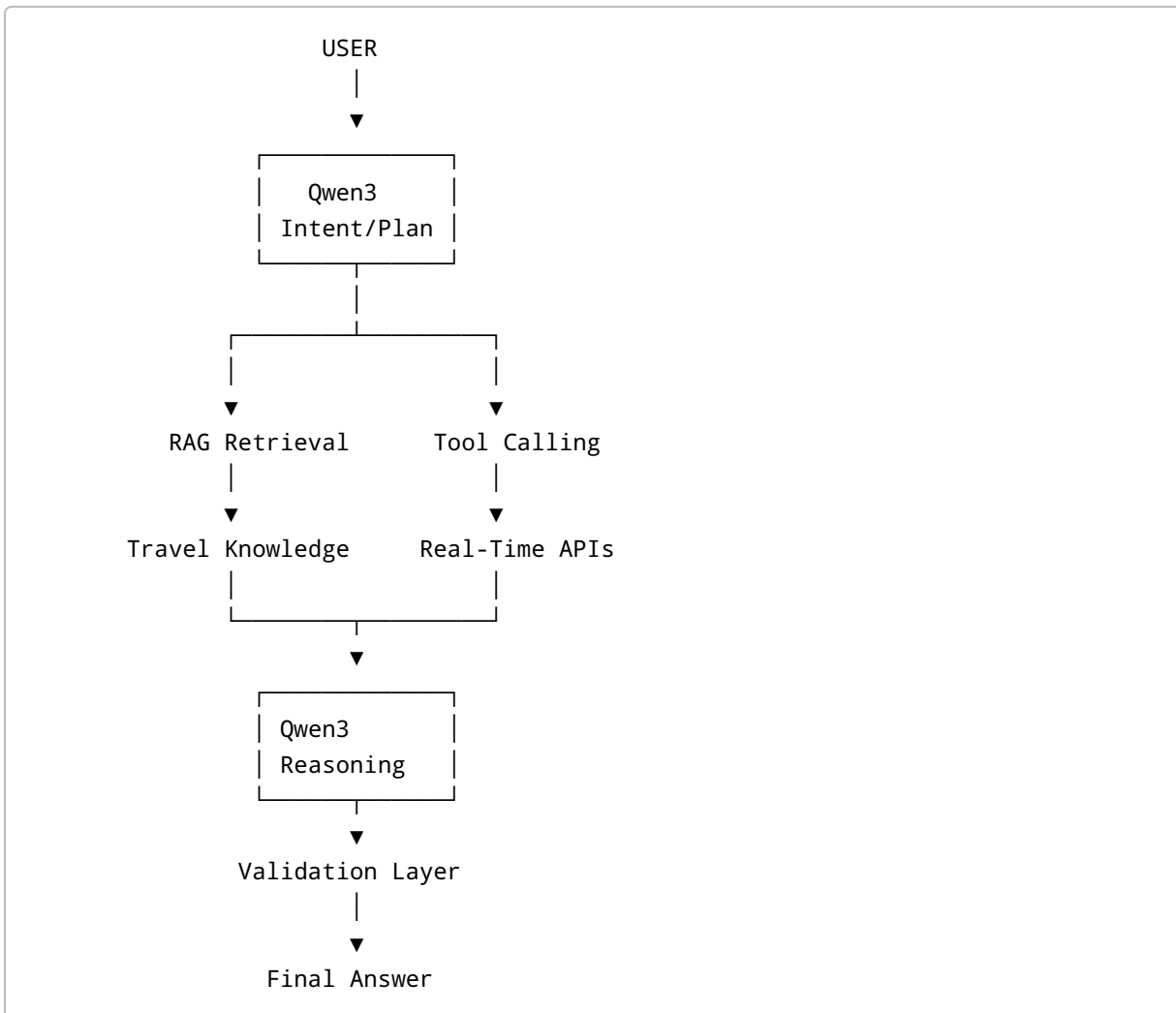
```

```
| database/
| models/
| schemas/
| middleware/
| config/
└─ requirements.txt
```

8. AI Architecture

The AI should NOT directly answer every question from its internal knowledge.

Use this architecture:



9. LLM Layer

Use **Qwen3** as the primary open-source model.

The LLM should be responsible for:

- Understanding user requests
- Extracting travel preferences
- Reasoning about destinations
- Selecting relevant tools
- Generating itinerary explanations
- Conversational interaction
- Summarizing retrieved information

The LLM should NOT be trusted as the source of:

- Current prices
- Current weather
- Current opening hours
- Current hotel availability
- Current routes
- Current booking availability

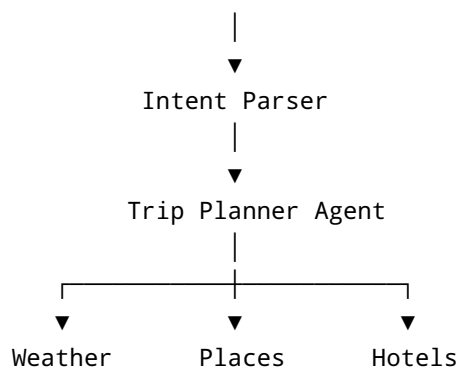
Those should come from verified APIs or your database.

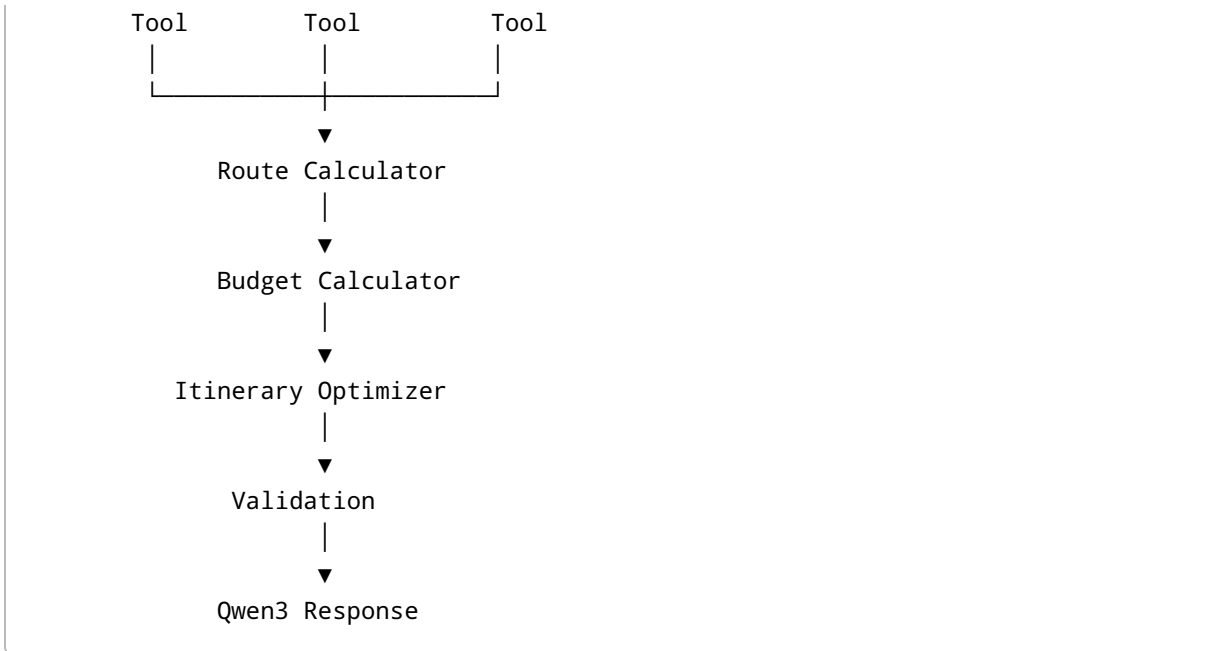
10. AI Agent Architecture

Use an agent/orchestrator layer.

Example:

User:
"Plan a 4-day trip to Ooty for ₹20,000."





11. Tool Calling Layer

Create specialized tools.

```
search_destinations()
search_places()
get_place_details()
get_weather()
calculate_route()
search_hotels()
search_restaurants()
search_activities()
calculate_budget()
get_opening_hours()
check_availability()
search_travel_knowledge()
```

The AI agent decides which tools to use.

12. RAG Architecture

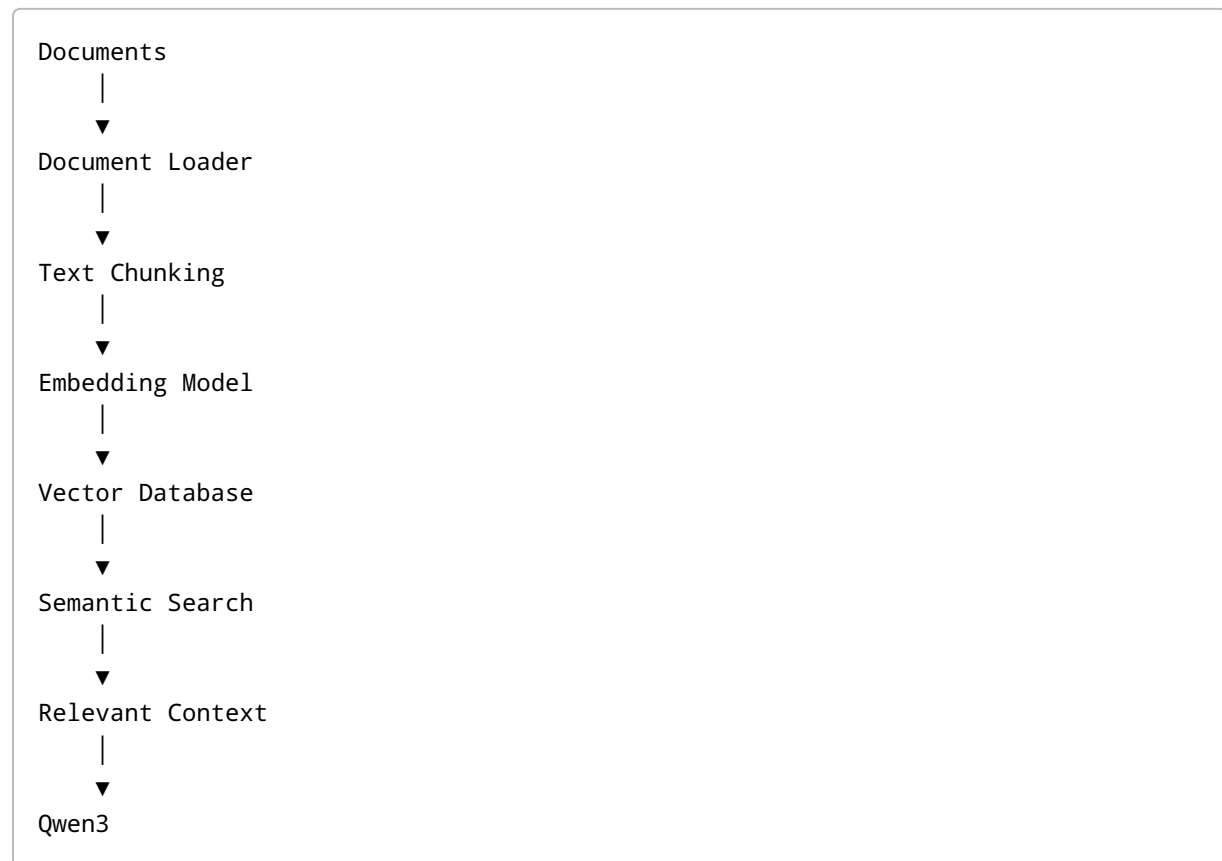
RAG should provide reliable travel knowledge.

Knowledge sources

Potential sources include:

- Official tourism websites
- Government tourism information
- Destination guides
- Cultural information
- Historical information
- Local rules
- Safety information
- Transportation information

Architecture:



Use **pgvector** if you want to keep vector data inside PostgreSQL, or Qdrant if you want a dedicated vector database.

13. Recommendation Engine

Don't depend entirely on the LLM for recommendations.

Create a recommendation scoring system.

Example:

```
Recommendation Score =  
  
0.25 × User Preference  
+ 0.20 × Budget Compatibility  
+ 0.15 × Distance  
+ 0.15 × Rating  
+ 0.10 × Weather Suitability  
+ 0.10 × Popularity  
+ 0.05 × Availability
```

The weights can later be learned from user behavior.

This gives you a more explainable recommendation system.

14. Itinerary Optimization

The itinerary engine should consider:

```
User Preferences  
    +  
Budget  
    +  
Opening Hours  
    +  
Distance  
    +  
Travel Time  
    +  
Weather  
    +  
Availability  
    ↓  
Optimized Itinerary
```

Example:

```
Hotel  
    ↓ 2 km
```

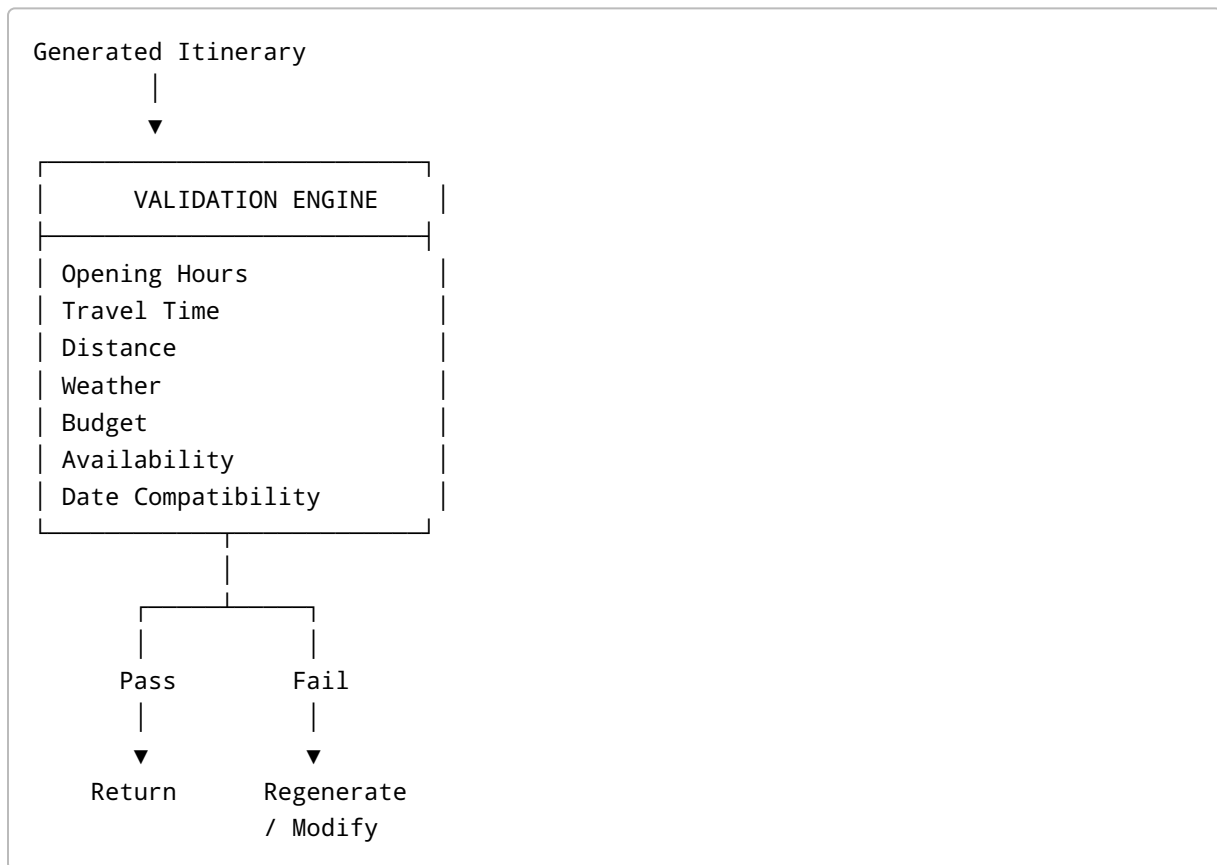
Attraction A
↓ 1.5 km
Restaurant
↓ 3 km
Attraction B

Avoid creating itineraries that require unnecessary travel.

15. Itinerary Validation

This is one of the most important components.

After Qwen3 generates the itinerary:

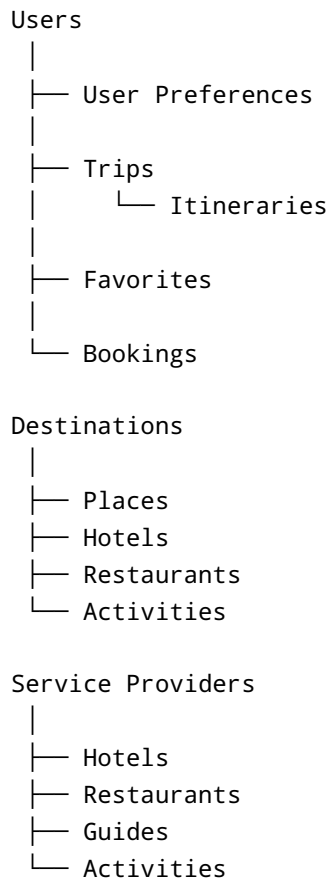


This significantly improves reliability.

16. Database Architecture

Use PostgreSQL.

Main entities:



17. Database Tables

Recommended initial tables:

```
users
user_preferences
destinations
places
hotels
restaurants
```

```
activities
service_providers
trips
itinerary_days
itinerary_items
favorites
reviews
bookings
notifications
ai_conversations
ai_messages
```

For RAG:

```
travel_documents
document_chunks
```

18. API Architecture

Use REST APIs initially.

Authentication

```
POST /api/auth/register
POST /api/auth/login
POST /api/auth/refresh
POST /api/auth/logout
```

Users

```
GET    /api/users/me
PUT    /api/users/me
GET    /api/users/preferences
PUT    /api/users/preferences
```

Destinations

```
GET /api/destinations
GET /api/destinations/{id}
GET /api/destinations/recommendations
```

Trips

```
POST /api/trips
GET /api/trips
GET /api/trips/{id}
PUT /api/trips/{id}
DELETE /api/trips/{id}
```

AI

```
POST /api/ai/chat
POST /api/ai/recommend
POST /api/ai/plan-trip
POST /api/ai/regenerate-itinerary
```

Hotels

```
GET /api/hotels
GET /api/hotels/{id}
```

Restaurants

```
GET /api/restaurants
GET /api/restaurants/{id}
```

Activities

```
GET /api/activities
GET /api/activities/{id}
```

19. Example AI Trip Planning API

Request

```
{
  "destination": "Ooty",
  "start_date": "2026-09-10",
  "end_date": "2026-09-13",
  "travelers": 4,
  "budget": 20000,
  "interests": [
    "nature",
    "food",
    "photography"
  ],
  "transportation": "car"
}
```

Processing

```
Request
  ↓
Validate Input
  ↓
Extract Preferences
  ↓
Search Places
  ↓
Check Weather
  ↓
Search Hotels
  ↓
Calculate Routes
  ↓
Calculate Budget
  ↓
Generate Itinerary
  ↓
Validate Itinerary
  ↓
Return Result
```

20. External API Layer

Do not connect external APIs directly from the frontend.

Use:

```
Frontend
  ↓
FastAPI
  ↓
External API Service Layer
```

Possible integrations:

```
Maps
Weather
Places
Hotels
Restaurants
Transportation
Payment
```

The service layer should normalize different API responses into your own internal format.

For example:

```
Google/OSM/Other API
  ↓
External Service Adapter
  ↓
Internal Place Schema
  ↓
Application
```

This makes it easier to replace APIs later.

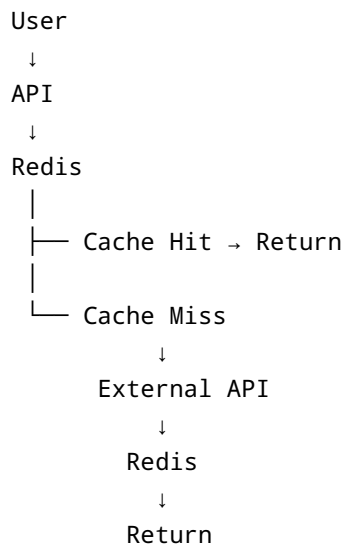
21. Caching Architecture

Use **Redis** for frequently requested data.

Cache:

- Weather
- Destination details
- Places
- Search results
- API responses
- Sessions
- Rate-limit counters

Example:



This reduces API cost and improves performance.

22. Authentication & Authorization

Use:

JWT
+
Refresh Tokens
+
Role-Based Access Control

Roles:

```
TOURIST
SERVICE_PROVIDER
ADMIN
```

Example:

```
Tourist
├─ Create trip
├─ Save destination
└─ Make booking

Service Provider
├─ Manage services
├─ Manage availability
└─ View bookings

Admin
├─ Manage users
├─ Manage providers
├─ Manage content
└─ Monitor platform
```

23. Security Architecture

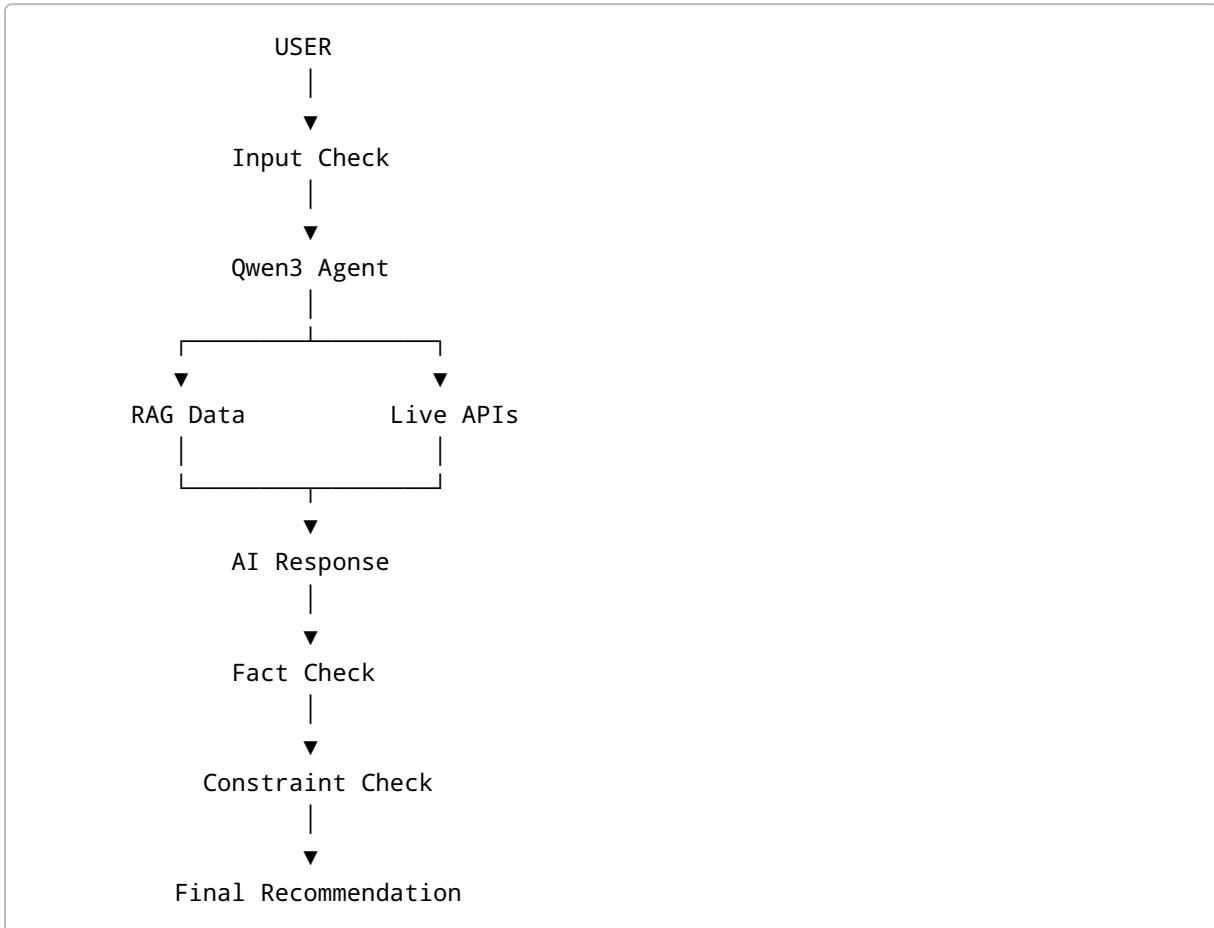
Implement:

- HTTPS
- Password hashing
- JWT authentication
- Role-based authorization
- API rate limiting
- Input validation
- SQL injection protection
- CORS configuration
- Secure HTTP headers
- API key protection
- Environment variables
- Logging and monitoring

Never place private API keys in the React application.

24. Reliability Architecture

The AI response should pass through multiple stages.



If required data cannot be verified:

Don't invent it.
Tell the user that the information could not be verified.

25. Observability

Implement logging and monitoring.

Monitor:

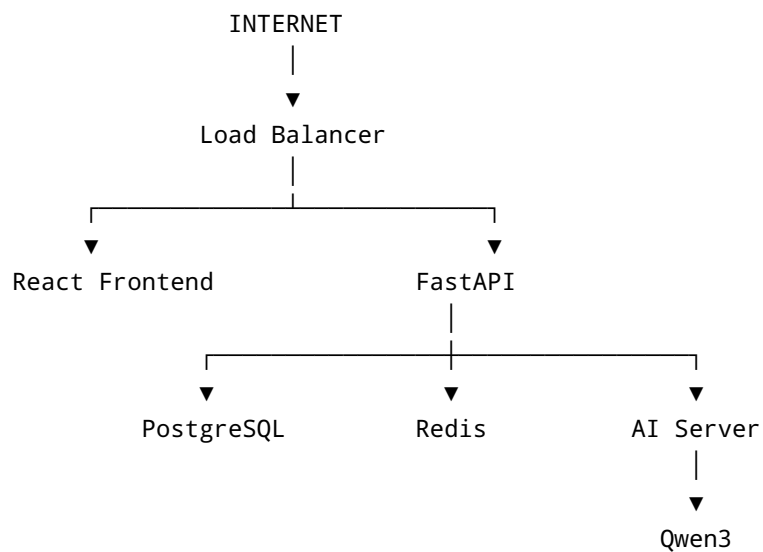
API latency
LLM latency
LLM token usage
API failures
Tool failures
Database performance
Cache hit rate
Recommendation quality
Error rate
User activity

Useful technologies:

Prometheus
Grafana
OpenTelemetry
Structured Logging

26. Deployment Architecture

For the first production version:



Containerize the components using Docker.

Example:

```
docker-compose.yml
```

```
frontend  
backend  
postgres  
redis  
qwen-server
```

27. AI Model Serving

For development:

```
Qwen3  
↓  
Ollama / vLLM  
↓  
FastAPI
```

For scalable deployment:

```
FastAPI  
↓  
AI Gateway  
↓  
vLLM  
↓  
GPU  
↓  
Qwen3
```

The model should be isolated from the main backend so that it can be scaled independently.

28. Development Roadmap

Phase 1 — Requirement Analysis

Define:

- User personas

- User journeys
- Functional requirements
- Non-functional requirements
- API requirements
- AI requirements
- Security requirements

Deliverables:

SRS
Use Case Diagram
User Stories
Requirement Matrix

Phase 2 — UI/UX

Create:

User Flow
Wireframes
Design System
High-Fidelity UI
Prototype

Main screens:

Landing
Explore
AI Planner
Trip Details
Hotels
Restaurants
Activities
Dashboard
Profile

Phase 3 — Database

Create PostgreSQL schema.

Implement:

```
Users
Trips
Destinations
Places
Services
Itineraries
Bookings
Reviews
```

Phase 4 — Backend

Build FastAPI services.

Start with:

```
Authentication
Users
Destinations
Trips
Places
```

Then add:

```
Hotels
Restaurants
Activities
```

Phase 5 — AI Foundation

Integrate Qwen3.

Build:

```
LLM Service
Prompt Manager
```

Structured Output
Conversation Memory

Phase 6 — RAG

Implement:

```
Document ingestion
↓
Chunking
↓
Embeddings
↓
Vector storage
↓
Retrieval
↓
Context injection
```

Phase 7 — Tool Calling

Implement:

```
Weather Tool
Places Tool
Maps Tool
Hotel Tool
Restaurant Tool
Budget Tool
```

Phase 8 — Recommendation Engine

Implement scoring based on:

```
Preferences
Budget
Distance
Rating
```

Weather
Availability
Popularity

Phase 9 — Itinerary Engine

Implement:

```
Place Selection
↓
Route Optimization
↓
Time Scheduling
↓
Budget Optimization
↓
Weather Validation
↓
Final Itinerary
```

Phase 10 — Frontend Integration

Connect:

```
React
↓
FastAPI
↓
AI Agent
↓
Database + External APIs
```

Replace mock data with real API responses.

29. MVP Scope

Do NOT attempt to build the entire platform initially.

Your MVP should contain:

- ✓ User registration
- ✓ User preferences
- ✓ Destination search
- ✓ AI destination recommendations
- ✓ AI trip planning
- ✓ Weather integration
- ✓ Places integration
- ✓ Route calculation
- ✓ Budget estimation
- ✓ Itinerary generation
- ✓ Save trip
- ✓ AI travel assistant

This is already a substantial project.

30. Version 2

After the MVP:

- ✓ Hotel recommendations
- ✓ Restaurant recommendations
- ✓ Activity recommendations
- ✓ Reviews
- ✓ Service providers
- ✓ Provider dashboard
- ✓ Notifications
- ✓ Better personalization

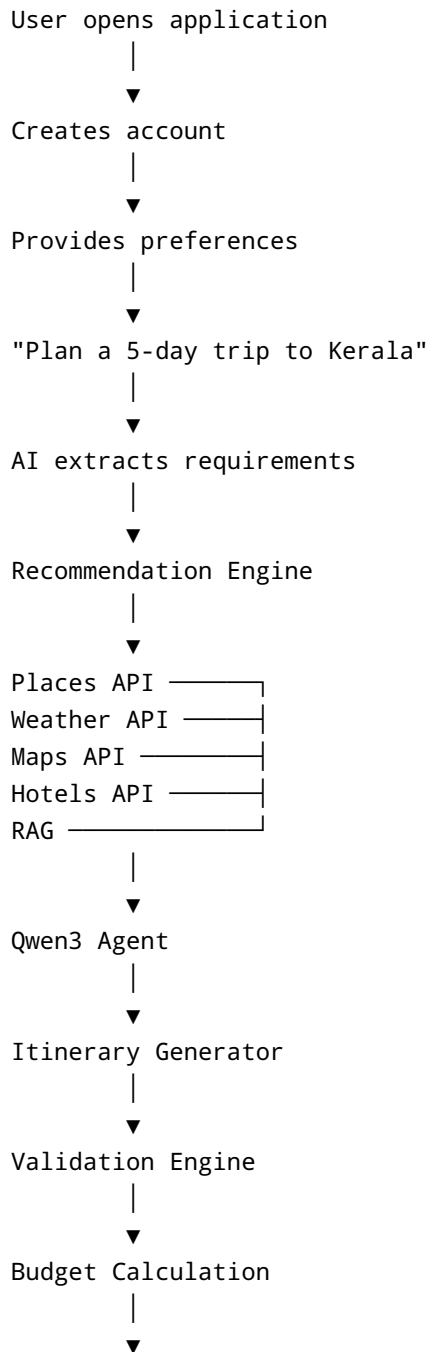
31. Version 3

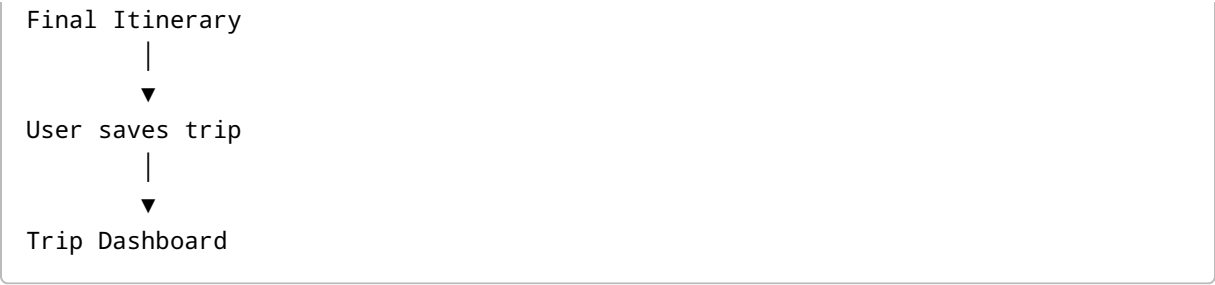
Advanced features:

- ✓ Hotel booking
- ✓ Activity booking
- ✓ Payment integration
- ✓ AI voice assistant
- ✓ Dynamic itinerary modification
- ✓ Real-time travel alerts

- ✓ Multi-language support
- ✓ User behavior learning
- ✓ Advanced recommendation models

32. Example Complete User Flow

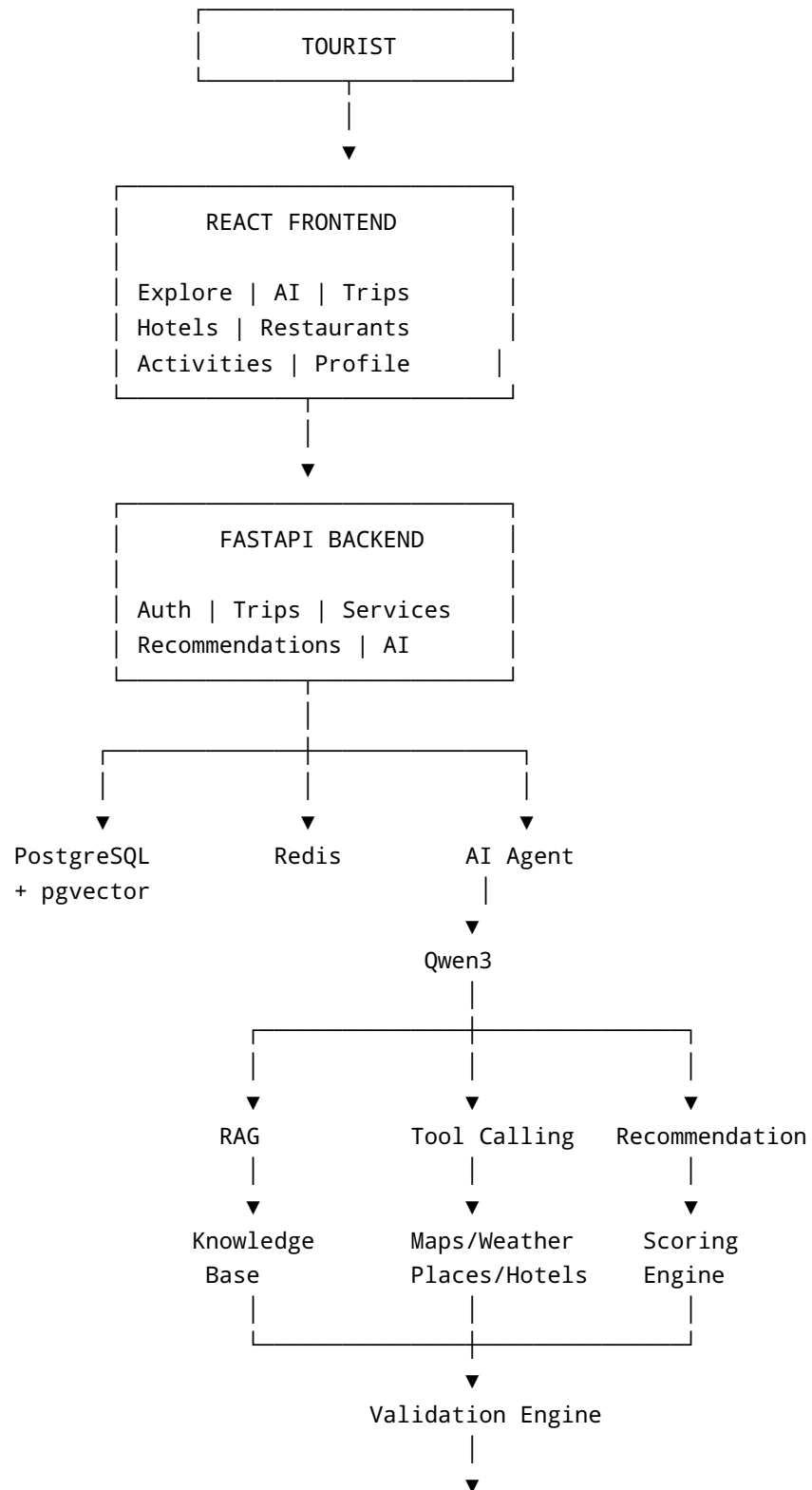


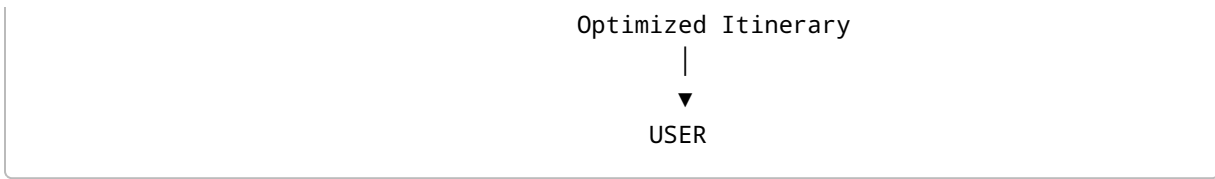


33. Recommended Final Technology Stack

Layer	Technology
Frontend	React + TypeScript
Styling	Tailwind CSS
Animation	Motion / Framer Motion
Backend	FastAPI
Language	Python
LLM	Qwen3
LLM Serving	vLLM
Agent	LangGraph / Qwen-Agent
RAG	pgvector / Qdrant
Database	PostgreSQL
Cache	Redis
Authentication	JWT/OAuth
Maps	Maps/OSM-based API
Weather	Weather API
Places	Places API
Containerization	Docker
Monitoring	Prometheus + Grafana
API Documentation	FastAPI / OpenAPI

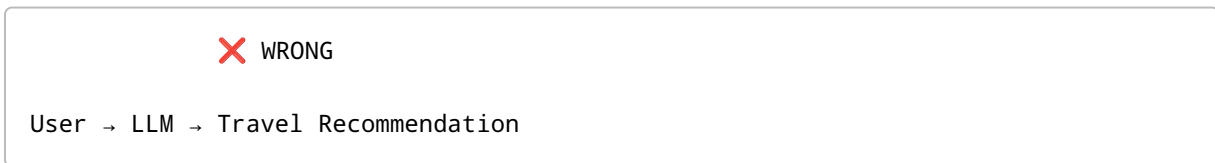
34. Final System Blueprint





35. Key Design Principle

The most important architectural decision is:



Instead, build:



This architecture makes the project substantially more reliable because the LLM is used for **reasoning and natural-language interaction**, while factual and time-sensitive information is obtained from appropriate data sources.

36. Development Priority

Follow this order:

1. Requirements
↓
2. UI/UX Design
↓
3. Database Design
↓
4. Backend APIs
↓
5. Authentication
↓
6. Qwen3 Integration
↓
7. RAG
↓
8. Tool Calling
↓
9. Recommendation Engine
↓
10. Itinerary Optimization
↓
11. Validation
↓
12. Frontend Integration
↓
13. Testing
↓
14. Deployment
↓
15. Monitoring

Final Architecture Goal

The finished product should not be just an **AI chatbot for travel**.

It should be an **AI Travel Agent platform** where:

Qwen3 = reasoning & conversation

RAG = trusted travel knowledge

External APIs = real-time information

Recommendation Engine = personalization

Optimization Engine = itinerary planning

Validation Engine = reliability

FastAPI = backend orchestration

React = premium user experience

PostgreSQL = persistent application data

Together, these components form the foundation of a scalable AI-powered travel recommendation and tourist service platform.